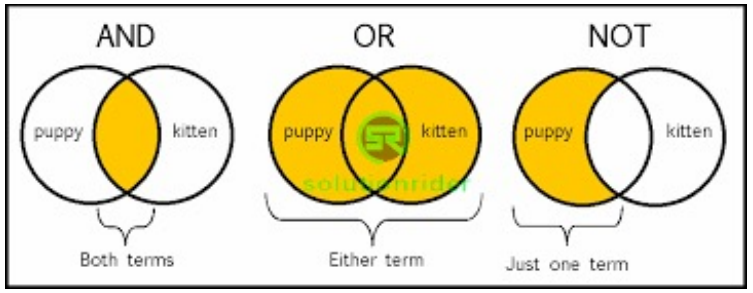
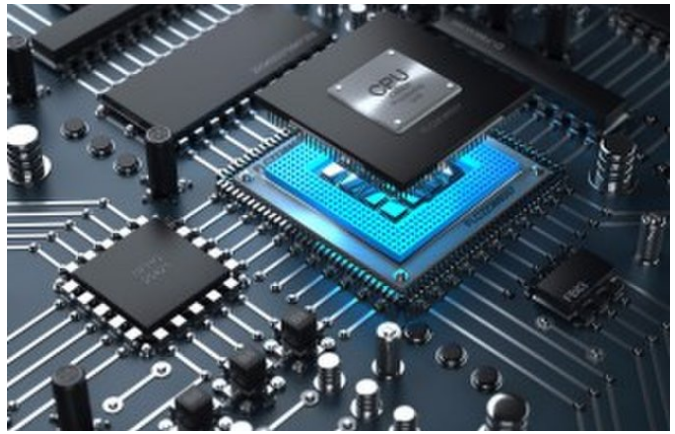




Lecture 03:
Binary Logic Operations

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Administrative

Lab Section: *How did last week's go?*
Current homework due **today** at 11:59 pm
New homework assignment will be available on Canvas – due next Monday!

On Canvas:

- **Office Hours** – See front page
- **Worksheet PDF** for today is available: *CS64_Lecture03_Worksheet.pdf*
- **New Reading** (for Wednesday + next week): *CS64_Handout2.pdf*

Remember: Quiz #1 is on Wednesday!

- **We will start it at 11 AM SHARP!!!**
- If you are late, you will not be allowed to take it (so, arrive early!)
- Material is from Lectures **1**, **2**, and **3**, and the **Reading** (CS64_Handout 1.pdf)

Worksheet

IMPORTANT: The use of **+** and **.** operators here are NOT the same as "addition" or "multiplication"!!! These are operators for LOGIC calculations, not arithmetic ones!

Binary Logic Refresher: NOT, AND, OR

inputs *output*

X	Y	X AND Y X.Y
0	0	0
0	1	0
1	0	0
1	1	1

AND
Logic Truth Table

input *output*

X	NOT X $\overline{X}$
0	1
1	0

NOT
Logic Truth Table

inputs *output*

X	Y	X OR Y X + Y
0	0	0
0	1	1
1	0	1
1	1	1

OR
Logic Truth Table

Worksheet

NOT AND (NAND)

X	Y	X AND Y X.Y	X NAND Y $\overline{X.Y}$
0	0	**0**	**1**
0	1	**0**	**1**
1	0	**0**	**1**
1	1	**1**	**0**

NOT OR (NOR)

X	Y	X OR Y X + Y	X NOR Y $\overline{X + Y}$
0	0	**0**	**1**
0	1	**1**	**0**
1	0	**1**	**0**
1	1	**1**	**0**

Worksheet

Exclusive-OR (XOR) and Exclusive-NOR (XNOR)

The output is “1” only if the 2 inputs are *opposites*

X	Y	$X \oplus Y$
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

The output is “1” only if the 2 inputs are the *same*

X	Y	$\overline{X \oplus Y}$
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**1**

Bitwise NOT

Similar to logical NOT (!), except it works on a **bit-by-bit** manner

In C/C++, it's denoted by a tilde operator: **~**

```
~(10011110) = 01100001
```

Worksheet

Exercises

What is ~(0x04)?

- Ans: 0xFB

```
  0x04      =  0000 0100
~(0x04)     =  1111 1011
            =  0xFB
```

What is ~(0xE7)?

- Ans: 0x18

```
  0xE7      =  1110 0111
~(0xE7)     =  0001 1000
            =  0x18
```

How is bitwise-NOT related to 2's Complement method?

Bitwise AND

Similar to logical AND (&&), except it works on a **bit-by-bit** manner

In C/C++, it's denoted by a <u>single</u> ampersand operator: **&**

```
(1001 & 0101)     =      1 0 0 1
                       & 0 1 0 1
                       = 0 0 0 1
```

Worksheet

Exercises

What is (0xFF) & (0x56)?

- Ans: 0x56

```
0xFF & 0x56=   1111 1111
           &   0101 0110
           =   0101 0110
           =   0x56
```

That's because ***1 & b = b***
And ***0 & b = 0***

What is (0x0F) & (0x56)?

- Ans: 0x06

What is (0x11) & (0x56)?

- Ans: 0x10

```
0x11 & 0x56=   0001 0001
           &   0101 0110
           =   0001 0000
           =   0x10
```

Bitwise OR

Similar to logical OR (||), except it works on a **bit-by-bit** manner

In C/C++, it's denoted by a <u>single</u> pipe operator: |

```
(1001 | 0101)     = 1 0 0 1
                  | 0 1 0 1
                  = 1 1 0 1
```

Worksheet

Exercises

What is (0xFF) | (0x92)?

- Ans: 0xFF

What is (0x00) | (0x92)?

- Ans: 0x92

That's because ***1 | b = 1***
And ***0 | b = b***

What is (0xA5) | (0x92)?

- Ans: 0xB7

```
0xA5 | 0x92=   1010 0101
           = | 1001 0010
           =   1011 0111
           =   0xB7
```

Bitwise XOR

X	Y	X ⊕ Y
0	0	0
0	**1**	**1**
1	**0**	**1**
1	1	0

Note:
(**1**^y) is always equal to **~y**
(**0**^y) is always equal to **y**

Works on a **bit-by-bit** manner

In C/C++, it's denoted by a single carat operator: **^**

```
(1001 ^ 0101) =    1 0 0 1
              ^    0 1 0 1
              =    1 1 0 0
```

Worksheet

Exercises

What is 0xFF ^ 0x13 *and also what's* 0x00 ^ 0x13?

- *Hint*: 1^b = ~b 0^b = b
- Ans: 0xEC and 0x13

What is 0xA1 ^ 0x13?

- Ans: 0xB2

```
0xA1 ^ 0x13=   1010 0001
             ^ 0001 0011
               1011 0010
             =  0xB2
```

Worksheet

iClicker!

What is **~(0xABC) & 0xFF0**?

A. 0x540
B. 0xAB0
C. 0x543
D. 0xFF0
E. 0x00C

0xABC = 1010 1011 1100
~(0xABC) = 0101 0100 0011

0xFF0 = 1111 1111 0000

Bitwise ANDing:
= 0101 0100 0000
= 0x540

Worksheet

iClicker!!

What is **~(0x37F | 0x124) ^ 0xF0F**?

A. 0xFFF

B. 0x38F

C. 0x37F

D. 0xC70

```
  0x37F   =   0011 0111 1111
| 0x124   =   0001 0010 0100
          =   0011 0111 1111
~(...)    =    1100 1000 0000

^ 0xF0F   =   1111 0000 1111
          =   0011 1000 1111
          =   0x38F
```

Worksheet

Bit Shift *Left*

Move all the bits N positions to the ***left***

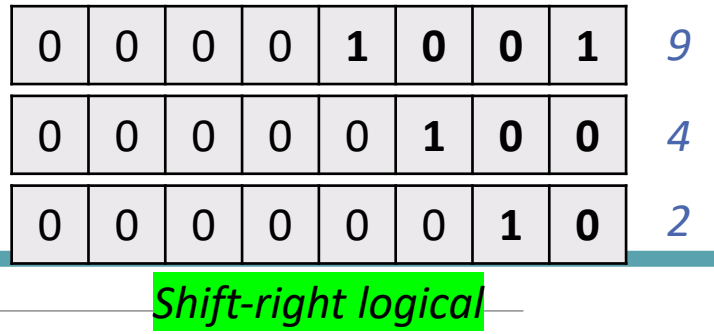
What do you do to the positions that are now empty? ANS: You put in 0s

Example: Shift "**1001**" 2 positions to the left **1001** << 2 = **100100**

Why is this useful as a form of *multiplication*?

Worksheet

Bit Shift *Right*



Move all the bits N positions to the ***right***

What do you do to the positions that are now empty? It DEPENDS!!!

Example: Shift "**1001**" 2 positions to the right:

1001 >> 2 = either **0010** or **1110**

(shift-right logical --- vs --- shift-right arithmetic)

Also note: the 2 least-significant bits are now ***lost*** (i.e., the orig. 2 bits on the right)

If Shift Left does *multiplication*, what does Shift Right do?

- It divides, **but** it truncates the result (a.k.a., integer division)

Two Forms of Shift Right

Subbing-in 0s in the MSBs makes sense (esp. if the number is unsigned)

1001 >> 2 = **0010**

BUT! If original MSBs are **1s** AND we are working with **signed** numbers, then we really should make the "new" MSBs still be 1s as well

1001 >> 2 = **1110**

- So, it keeps the result negative: This is also known as ***sign extension***

BUT WAIT!! What if it's a **signed** number BUT it's positive?

- ANS: ***Sign extension*** again! You should sub-in the leftmost bits with 0s!

0100 >> 2 = **0001**

This is called ***"arithmetic"*** shift right and it's the default shift-right op in C/C++

- **Arithmetic** shift *extends* the sign bit (*sign extension*, aka, *sign preservation*)

Worksheet

iClicker!!!

What is $(15)_{10}$ << 2?

A. 1500

B. 150

C. 60

D. 30

E. 15

15 = 01111 in binary

(note, I put a 0 as MSB to emphasize that this is +15)

01111 << 2 = 0111100

Equivalent to:

$2^5 + 2^4 + 2^3 + 2^2$ = 32+16+8+4 = 60

***OR*:**

Realize that << 2 is equiv. to multiply by 2^2

So, the answer is 15 x 4 = 60

Worksheet

iClicker!!!!

What is $(-15)_{10}$ << 2?

A. –1500
B. –68
C. –60
D. –30
E. –15

–15 = 10001 in binary *(take 2's comp. of 01111)*

10001 << 2 = 1000100

Equivalent to: –(0111100) *(take 2's comp.)*

$-(2^5 + 2^4 + 2^3 + 2^2) = -(32+16+8+4) = \underline{-60}$

***OR*:**

Again, realize that << 2 is equiv. to x2^2

So, the answer is –15x4 = <u>–60</u>

Worksheet

iClicker!!!!!

What is $(\mathbf{15})_{10}$ **>> 2**?

A. 15

B. 7

C. 3

D. 1

E. 0

15 = 01111 in binary (note, I put a 0 as MSB)

01111 >> 2 = 011 = $(\mathbf{3})_{10}$

***OR*:**

Realize that >> 2 is equiv. to int divide by 2^2
So, the answer is 15//4 = 3

Worksheet

Class Exercise: In C++

The following C++ code will print out bit-wise logic operation results:

```
int a = 255, b = 16;     // note these are decimal numbers
cout << (a & b) << endl;      // prints 16
cout << (a | b) << endl;      // prints 255
cout << (a ^ b) << endl;      // prints 239
cout << (a >> 4) << endl;     // prints 15
cout << (b << 3) << endl;     // prints 128
```

Worksheet

Exercise Using Logic Ops

Given an argument that's a 32-bit integer number **i**, ***write a function in C++*** that can isolate the bit in **position 5** of that integer and print it.

Example: **i** = 1266

In 32-bits of binary, that's:

the bit in position 5 is the highlighted one

0000 0000 0000 0000 0000 0100 1111 0010

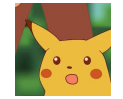
So, your function should print out "**1**" for this input **i**

```
void print5(int i) {
    i = i >> 5;      // moves bit5 into the bit0 (LSB) position
    i = i & 1;       // makes all bits 0, except the LSB
    cout << i;
}
```

Worksheet

Bit Shifting in CPUs

```
int a = 3;
for(int i = 0; i < 40; i++){
    a = a << 1;
    cout << a << endl;    }
```

	0	0	0	0	0	… 0 …	0	0	1	1	3
bit position	*31*	*30*	*29*	*28*	*27*		*3*	*2*	*1*	*0*	
0	0	0	0	0	0	… 0 …	0	1	1	0	6
<< 1	*31*	*30*	*29*	*28*	*27*		*3*	*2*	*1*	*0*	
<< 1	0	1	1	0	0	… 0 …	0	0	0	0	$2^{30} + 2^{29}$
many more times	*31*	*30*	*29*	*28*	*27*		*3*	*2*	*1*	*0*	
<< 1	1	1	0	0	0	… 0 …	0	0	0	0	-2^{30} 
	31	*30*	*29*	*28*	*27*		*3*	*2*	*1*	*0*	
<< 1 twice	0	0	0	0	0	… 0 …	0	0	0	0	0
	31	*30*	*29*	*28*	*27*		*3*	*2*	*1*	*0*	

```cpp
#include <iostream>
using namespace std;

int main() {

int a = 3;
for(int i = 0; i < 40; i++){
    a = a << 1;     // shift-left by 1 bit
    cout << "SLL by " << i+1 << " bit(s) : " << a << endl;    }

return 0;
}
```

To further illustrate the previous slide…

```
SLL by 1 bit(s) : 6
SLL by 2 bit(s) : 12
SLL by 3 bit(s) : 24
SLL by 4 bit(s) : 48
SLL by 5 bit(s) : 96
SLL by 6 bit(s) : 192
SLL by 7 bit(s) : 384
SLL by 8 bit(s) : 768
SLL by 9 bit(s) : 1536
SLL by 10 bit(s) : 3072
SLL by 11 bit(s) : 6144
SLL by 12 bit(s) : 12288
SLL by 13 bit(s) : 24576
SLL by 14 bit(s) : 49152
SLL by 15 bit(s) : 98304
SLL by 16 bit(s) : 196608
SLL by 17 bit(s) : 393216
SLL by 18 bit(s) : 786432
SLL by 19 bit(s) : 1572864
SLL by 20 bit(s) : 3145728
SLL by 21 bit(s) : 6291456
SLL by 22 bit(s) : 12582912
SLL by 23 bit(s) : 25165824
SLL by 24 bit(s) : 50331648
SLL by 25 bit(s) : 100663296
SLL by 26 bit(s) : 201326592
SLL by 27 bit(s) : 402653184
SLL by 28 bit(s) : 805306368
SLL by 29 bit(s) : 1610612736
SLL by 30 bit(s) : -1073741824
SLL by 31 bit(s) : -2147483648
SLL by 32 bit(s) : 0
SLL by 33 bit(s) : 0
SLL by 34 bit(s) : 0
SLL by 35 bit(s) : 0
SLL by 36 bit(s) : 0
SLL by 37 bit(s) : 0
SLL by 38 bit(s) : 0
SLL by 39 bit(s) : 0
SLL by 40 bit(s) : 0
```

YOUR TO-DOs

Check Piazza every day! ☺

Turn-in current homework assignment today by 11:59 PM

Look for new homework assignment on Canvas

Quiz #1 on Wednesday!!

Office hours!

</LECTURE>